

JAVASCRIPT NOTES

</>



JS PLACEMENT

JavaScript code can be placed anywhere in an HTML document, but commonly it is included using the `<script>` tag in the following ways.

Ways to Include JavaScript in HTML

1. Inside `<head>` section
2. Inside `<body>` section
3. In both `<head>` and `<body>` sections
4. In an external `.js` file and linked to HTML

Basic syntax:

```
<script>
  // JavaScript code
</script>
```

1. JavaScript in `<head>` Section

- Used when JavaScript should execute on user actions (events like button click).
- Functions are defined in the `<head>` and executed later.

Use case: Event-driven scripts

✓ Example: Button click alert

2. JavaScript in `<body>` Section

- Used when JavaScript should run while the page loads.
- Directly writes or modifies page content.

Use case: Dynamic content generation

✓ Example: `document.write()`

3. JavaScript in Both `<head>` and `<body>`

- Functions are defined in `<head>`.
- Code execution or output is done in `<body>`.

Use case: Combined event handling and page rendering

4. JavaScript in External File

- JavaScript code is written in a separate `.js` file.
- Included in HTML using the `src` attribute.
- External JS files do not use `<script>` tags inside them.

Advantages:

- Code reusability
- Better maintenance
- Cleaner HTML

```
<script src="filename.js"></script>
```

Including Multiple JavaScript Files

Multiple scripts can be added using multiple `<script>` tags:

```
<script src="file1.js"></script>
```

```
<script src="file2.js"></script>
```

Ways to Add External JavaScript Files

1. Full file path

```
<script src="C://javascript/filename.js"></script>
```

2. Relative path

```
<script src="javascript/filename.js"></script>
```

3. Same folder

```
<script src="file.js"></script>
```

Advantages of Using `<script>` Tag

- Easy integration of JavaScript into HTML
- Immediate execution when browser encounters the script
- Supports inline and external scripts
- Allows integration of external libraries/frameworks (e.g., jQuery via CDN)
- Provides global scope access to variables and functions

Example of external library inclusion:

```
<script src="https://cdnjs.cloudflare.com/ajax/libs/jquery/3.7.0/jquery.min.js"></script>
```

JS SYNTAX

JavaScript syntax defines the rules for writing valid JavaScript code. JavaScript code is written inside the `<script>...</script>` tag in an HTML document, usually placed in the `<head>` section, though it can appear anywhere in the page.

`<script>` Tag

- Tells the browser to interpret the content as JavaScript.
- Earlier attributes:
 - `language="javascript"` (now deprecated)
 - `type="text/javascript"` (optional in HTML5)
- Modern browsers treat JavaScript as the default scripting language.

First JavaScript Program

- `document.write()` is used to display content on the web page.
- It can output text, HTML, or both.

Example output:

Hello World!

JavaScript Values

JavaScript supports two types of values:

1. Literals (Fixed values) – Numbers, strings
2. Variables (Dynamic values) – Store changing data

JavaScript Literals

- Number literal: 10
- String literal: "Hello"

JavaScript Variables

Used to store data dynamically.

Keywords used:

- **var**
- **let**
- **const**

Values are assigned using the = operator.

Whitespace and Line Breaks

- JavaScript **ignores spaces, tabs, and line breaks.**
- Proper indentation improves readability but does not affect execution.

Semicolons

- **Semicolons are optional when statements are on separate lines.**
- **Mandatory when multiple statements appear on the same line.**
- Using semicolons is considered good programming practice.

Case Sensitivity

- JavaScript is **case-sensitive**.
- Identifiers like time and Time are treated as different.
- Incorrect capitalization can lead to errors.

Variable Naming Conventions

- **Pascal Case: SmartWatch**
- **Camel Case (preferred): smartWatch**
- **Underscore: smart_watch**
- **Hyphens (-) are not allowed in identifiers.**

JavaScript Keywords

- Reserved words used for specific purposes.
- Examples: function, var, let, const
- Keywords cannot be used as variable or function names.

JavaScript Identifiers

Identifiers are names of variables, functions, and objects.

Rules:

- **Must start with a letter, _, or \$**
- Cannot start with a digit or hyphen
- Can contain digits after the first character

Comments in JavaScript

Used to explain code and improve readability.

- **Single-line:** `// comment`
- **Multi-line:** `/* comment */`
- HTML comment opening `<!--` is treated as a single-line comment

Operators in JavaScript

JavaScript supports:

- Arithmetic
- Logical
- Assignment
- Bitwise operators
- `+` performs string concatenation if one operand is a string.

Expressions in JavaScript

An expression combines values, variables, and operators.

Examples:

- `10 + 20`
- `a * b`
- `c / 2`

Expressions can be arithmetic or assignment expressions.

JavaScript Character Set

- Uses Unicode character set
- Supports symbols, emojis, and special characters
- Example:
 - `←` → ←
 - `₹` → Rupee symbol

HELLO WORLD PROGRAM

A Hello World program is generally used to demonstrate the basic syntax of a programming language. In JavaScript, there are multiple ways to display the output "Hello World".

1. Using `document.write()`

The `document.write()` method writes content directly to the HTML document.

Syntax

`<script>`

```
document.write("Hello World");  
</script>
```

Points to Remember

- Must be written inside <script> tags
- Can be placed inside <head> or <body>
- Mainly used for learning purposes
- Not recommended for modern applications

Example

```
<html>  
<head>  
  <script>  
    document.write("Hello World");  
  </script>  
</head>  
<body>  
</body>  
</html>
```

2. Using alert() Method

The alert() method displays output in a popup dialog box.

Syntax

```
<script>  
  alert("Hello World");  
</script>
```

Notes

- alert() is a window method
- window.alert() and alert() are the same
- Blocks execution until user clicks OK

Example

```
<html>  
<head>  
  <script>  
    alert("Hello World");  
  </script>  
</head>  
<body>  
</body>  
</html>
```

3. Using console.log()

The console.log() method prints output to the browser console.

It is widely used for debugging.

Syntax

```
console.log("Hello World");
```

Example with HTML

```
<html>
<head>
  <script>
    console.log("Hello World");
  </script>
</head>
<body>
  <p>Open the browser console to see the output</p>
</body>
</html>
```

Output in Console

Hello World

4. Using innerHTML

The innerHTML property is used to change the content of an HTML element.

Steps

1. Access the element using getElementById()
2. Assign value to innerHTML

Example

```
<html>
<head>
  <title>Using innerHTML</title>
</head>
<body>
  <div id="output"></div>

  <script>
    document.getElementById("output").innerHTML = "Hello World";
  </script>
</body>
</html>
```

JavaScript console.log() Method

The console.log() method is used to print messages or values in the web console.

Syntax

```
console.log(msg1, msg2, ..., msgN);
```

You can pass multiple values separated by commas.

Example: Printing Variables

```
<html>
<head>
  <title>console.log Example</title>
</head>
<body>
  <script>
    console.log("Hello World!");
    var num1 = 30;
    var num2 = 20;
    console.log("The sum of", num1, "and", num2, "is:", num1 + num2);
  </script>
</body>
</html>
```

Console Output

Hello World!

The sum of 30 and 20 is: 50

Example: Printing an Object

```
<html>
<head>
  <title>console.log Object Example</title>
</head>
<body>
  <script>
    var obj = {
      website: "Tutorialspoint",
      domain: "www.tutorialspoint.com",
      description: "This site is for learning."
    };
    console.log(obj);
  </script>
</body>
</html>
```


Console Output

```
{website: 'Tutorialspoint', domain: 'www.tutorialspoint.com', description: 'This site is for learning.'}
```

Console.log() with Server-Side JavaScript (Node.js)

In Node.js, console.log() prints output in the terminal.

Example (app.js)

```
let message = "Hello World!";  
console.log(message);
```

Run Command

```
node app.js
```

Output

```
Hello World!
```

Summary Table

Method	Output Location	Usage
document.write()	Web Page	Basic learning
alert()	Popup Box	Notifications
console.log()	Browser Console	Debugging
innerHTML	Web Page Element	Dynamic content

JavaScript Variables

JavaScript variables are used to **store data values**. A variable can be thought of as a **named container** that holds data, which can be accessed and modified during program execution.

Declaring Variables in JavaScript

JavaScript allows variables to be declared in **four ways**:

1. **Without using any keyword**
2. **Using var**
3. **Using let**
4. **Using const**

let and const were introduced in **ES6 (2015)**. Before ES6, only var was used.

1. Variable Declaration Without Keyword

```
<script>
  money = 10;
  name = "tutorialspoint";
</script>
```

Note:

- Variables declared without a keyword become **global variables**
- **This is not recommended in modern JavaScript**

2. Variable Declaration Using var Declaration

```
<script>
  var money;
  var name;
</script>
```

Multiple Declarations

```
<script>
  var money, name;
</script>
```

Variable Initialization

Assigning a value to a variable is called **initialization**.

```
<script>
  var name = "Ali";
  var money;
  money = 2000.50;
</script>
```

Important Rule

- A variable should be **declared only once**
- Re-declaration with var is allowed but **not recommended**

JavaScript is Dynamically Typed

JavaScript is an **untyped (dynamically typed)** language.

```
<script>
  var num = 765;    // Number
  var str = "Welcome"; // String
  var bool = false; // Boolean
</script>
```

A variable's **type can change at runtime**.

JavaScript Variable Names (Identifiers)

Rules

- Can contain letters, digits, _, \$
- Cannot start with a digit
- Case-sensitive (Name $\neq$ name)
- Unicode characters are allowed
- Reserved keywords are **not allowed**

Example

```
<script>
  var _abc = "Hi!";
  var $abc = "Hello!";
  // var 9abc = "Bye!"; // Invalid
</script>
```

Undefined Variables

If a variable is declared but **not initialized**, its value is undefined.

```
<script>
  var num;
  document.write(num);
</script>
```

Output

undefined

JavaScript Variable Scope

JavaScript variables have **two scopes**:

1. Global Scope

- Declared outside any function
- Accessible everywhere

2. Local (Function) Scope

- Declared inside a function
- Accessible only within that function

Example: Global vs Local Variable

```
<script>
  var myVar = "global";
  function checkscope() {
    var myVar = "local";
    document.write(myVar);
  }
  checkscope();
```

```
</script>
```

Output

local

Variables Without var Are Global

```
<script>
  name = "tutorialspoint";
  number = 10.25;
</script>
```

Avoid this practice

Re-declaration with var

```
<script>
  var age = 10;
  var age;
  document.write(age);
</script>
```

Output

10

JavaScript let Statement

The let keyword declares a **block-scoped variable**.

```
let name = "John";
let age = 35;
```

Block Scope vs Function Scope

Block Scope (let)

```
{
  let x = 10;
}
// x is NOT accessible here
```

Function Scope (var)

```
function test() {
  if (true) {
    var y = 20;
  }
  // y is accessible here
}
```

Example: let vs var

```
<script>
  let x = 10;
  var y = 20;

  function test() {
    let x = 50;
    var y = 100;
    document.write("x = " + x + ", y = " + y);
  }
  test();
</script>
```

Output

x = 50, y = 100

Explanation (brief and precise):

- let x = 10 and var y = 20 are global variables.
- Inside test():
 - let x = 50 creates a block-scoped local variable that shadows the global x.
 - var y = 100 creates a function-scoped local variable that shadows the global y.
 - document.write() accesses the local variables inside the function, not the globals.

Redeclaring Variables

Keyword	Redeclare Allowed
var	Yes
let	No
const	No

let m = 70;

// let m = 80; invalid

var n = 80;

var n = 90; valid

Variable Hoisting

- var → Hoisted (initialized as undefined)
- let → Not hoisted
- const → Not hoisted

```
n = 50;  
var n; // valid
```

```
x = 10;  
let x; // error
```

JavaScript const Keyword

The const keyword is used to declare **constant references**.

```
const x = 10;
```

Rules

- Must be initialized
 - Cannot be reassigned
 - Block-scoped
 - Not hoisted
-

Constant Arrays

```
<script>  
  const arr = ["door", "window"];  
  arr[0] = "gate";  
  arr.push("roof");  
  document.write(arr);  
</script>
```

```
arr = [] (reassignment not allowed)
```

Constant Objects

```
<script>  
  const obj = { animal: "Lion" };  
  obj.animal = "Tiger";  
  obj.legs = 4;  
  document.write(JSON.stringify(obj));  
</script>
```

Difference Between var, let, and const

Feature	var	let	const
Scope	Function	Block	Block
Hoisting	Yes	No	No
Reassign	Yes	Yes	No
Redeclare	Yes	No	No
Initialization Required	No	No	Yes

Which One Should You Use?

- Use **let** for normal variables
- Use **const** for fixed values
- Avoid **var** unless required
- Avoid variables without keywords

JavaScript Data Types

1. JavaScript Data Types

Data types define the kind of values a variable can store and how operations are performed on them. JavaScript is a **dynamically typed** language, meaning variable types are decided at runtime.

2. Categories of Data Types

A. Primitive Data Types (7)

Introduced in ES6 and later:

1. **String** – Text values
 - Created using `'`, `"`, or ```
2. **Number** – Integer and floating-point numbers (IEEE 754, 64-bit)
3. **Boolean** – true or false
4. **null** – Represents an intentional empty or unknown value
5. **undefined** – Variable declared but not assigned a value
6. **BigInt** – Used for very large integers (n suffix)
7. **Symbol** – Unique and immutable identifiers (ES6)

In ES5, only 5 primitive types existed (String, Number, Boolean, null, undefined).

Example

```
/*primitive Data Types
1.string
2.number - 1,2,3
3.null
4.boolean - true or false
5.bigint - 12345678
6.symbol
7.undefined
*/

let a="Anila";
let age=28;
let not=null;
let bol=true;
let big=1234567890;
let sym=Symbol("ph");
let notdefined;

console.log(a,age,not,bol,big,sym,notdefined);
```

B. Non-Primitive (Reference) Data Type

- **Object** – Stores data in key–value pairs
- **Array** – Ordered list of values
- **Date** – Date and time manipulation

Example

```
/*
Non-Primitive (Reference) Data Type
Object – Stores data in key–value pairs
Array – Ordered list of values
Date – Date and time manipulation
*/

let fruits={name:"apple",color:"red"};
let arr=[1,2,3];
let today=new Date();
console.log(fruits,arr,today);
```

3. Why Data Types Matter

- JavaScript performs **automatic type conversion**, which can lead to unexpected results.
 - Example:
let sum = "10" + 10; // "1010"
Here, number 10 is converted to a string.
-

4. typeof Operator

Used to check the data type of a variable:

```
typeof 10      // "number"  
typeof "Hello" // "string"  
typeof true    // "boolean"
```

Example

```
//data type checking - typeof  
// typeof null - object will come - (this is a known quirk in JavaScript)  
console.log(typeof a,typeof age,typeof not,typeof bol,typeof big,typeof sym,typeof notdefined);  
console.log(typeof fruits,typeof arr,typeof today);
```

5. Dynamic Typing

A variable can hold different data types at different times:

```
let x = "One";  
x = 1;  
x = true;
```

Example

```
// Example of variable reassignment  
let score = 100;  
console.log(score); // 100  
  
score = 150; // valid reassignment  
console.log(score); // 150  
  
const pi = 3.14;  
//pi = 3.14159; // invalid, will throw an error if uncommented  
console.log(pi); // 3.14
```

JavaScript Type Conversion

Type conversion means changing a value from one data type to another.

A. Implicit Type Conversion (Coercion)

Done automatically by JavaScript.

- **String Conversion**

Occurs when + operator is used with a string:

"100" + 24 // "10024"

"100" + null // "100null"

- **Number Conversion**

Occurs with arithmetic operators except +:

"100" - 50 // 50

"100" * true // 100

"tp" / 50 // NaN

- **Boolean Conversion**

Using !!:

!!0 // false

!!1 // true

!!"" // false

!!"Hi" // true

Special Cases

- null → 0 in numeric operations
- undefined → NaN in numeric/boolean operations

Example:

```
//implicit
//string conversion
a= 10+"24";
console.log(a);
//number conversion
a= "10"-24;
b=10/5;
c="100"*true
console.log(a,b,c);
//boolean conversion
a= !!0;
b=!!1;
c=!!"";
d=!!"HI"
console.log(a,b,c,d);
```

B. Explicit Type Conversion

Done manually using built-in functions.

- **To String**

```
let num=String(100)
let notnulldefined=String(null)
console.log(num.toString(),notnulldefined.toString());
```

- **To Number**

```
//tonumber
let anum= Number("100");
let bnum=parseInt("100");
let cnum=parseFloat("10.5")+200;
console.log(anum,bnum,cnum);
```

- **To Boolean**

```
let abool=Boolean(0) ;    // false
let bbool=Boolean("")    // false
let cbool=Boolean("Hello") // true
console.log(abool,bbool,cbool);
```

C. Date Conversion

- **Date → Number**: milliseconds since 1-Jan-1970

```
let date = new Date();
let value = date.getTime();
console.log(date, value);
```

- **Date → String**

```
//date-> string
let date1 = new Date();
console.log(String(date1));
```

8. Common Conversion Results (Key Takeaways)

Value	String	Number	Boolean
0	"0"	0	false

"0"	"0"	0	true
""	""	0	false
"Hello"	"Hello"	NaN	true
null	"null"	0	false
undefined	"undefined"	NaN	false
true	"true"	1	true
false	"false"	0	false

Final Note

- JavaScript's flexibility with types is powerful but error-prone.
- Always be cautious with **implicit conversions**.
- Prefer **explicit conversions** for clarity and reliability, especially in comparisons and calculations.

EXAMPLE

```

/*primitive Data Types
1.string
2.number - 1,2,3
3.null
4.boolean - true or false
5.bigint - 12345678
6.symbol
7.undefined
*/

let a="Anila";
let age=28;
let not=null;
let bol=true;
let big=1234567890;
let sym=Symbol("ph");
let notdefined;

console.log(a,age,not,bol,big,sym,notdefined);

/*

```

Non-Primitive (Reference) Data Type

Object – Stores data in key–value pairs

Array – Ordered list of values

Date – Date and time manipulation

```
*/  
  
let fruits={name:"apple",color:"red"};  
let arr=[1,2,3];  
let today=new Date();  
  
console.log(fruits,arr,today);  
  
//data type checking - typeof  
// typeof null - object will come - (this is a known quirk in JavaScript)  
console.log(typeof a,typeof age,typeof not,typeof bol,typeof big,typeof sym,typeof notdefined);  
console.log(typeof fruits,typeof arr,typeof today);  
  
// Example of variable reassignment  
let score = 100;  
console.log(score); // 100  
  
score = 150; // valid reassignment  
console.log(score); // 150  
  
const pi = 3.14;  
//pi = 3.14159; // invalid, will throw an error if uncommented  
console.log(pi); // 3.14  
  
/*  
type conversion  
implicit conversion(done automatically)  
explicit conversion(done manually using built in functions)  
*/  
  
//implicit  
//string conversion  
a= 10+"24";  
console.log(a);  
  
//number conversion  
a= "10"-24;  
b=10/5;
```

```
c="100"*true
console.log(a,b,c);

//boolean conversion
a= !!0;
b=!!1;
c=!!"";
d=!!"HI"
console.log(a,b,c,d);

//explicit
//toString
let num=String(100)
let notnulldefined=String(null)
console.log(num.toString(),notnulldefined.toString());

//tonumber
let anum= Number("100");
let bnum=parseInt("100");
let cnum=parseFloat("10.5")+ "200";
console.log(anum,bnum,cnum);

//toboolean
let abool=Boolean(0) ;    // false
let bbool=Boolean("")    // false
let cbool=Boolean("Hello") // true
console.log(abool,bbool,cbool);

// date conversion
// date -> number
let date = new Date();
let value = date.getTime();
console.log(date, value);

let value1 = Date.now();
console.log(value1);

let value2 = Number(new Date());
console.log(value2);
```

```
//date-> string
let date1 = new Date();
console.log(String(date1));
```

OUTPUT

```
PS C:\Users\anila\Desktop\Course Study\14. JS> node .\datatype.js
Anila 28 null true 1234567890 Symbol(ph) undefined
{ name: 'apple', color: 'red' } [ 1, 2, 3 ] 2026-01-30T09:05:57.540Z
string number object boolean number symbol undefined
object object object
100
150
3.14
1024
-14 2 100
false true false true
100 null
100 100 10.5200
false false true
2026-01-30T09:05:57.546Z 1769763957546
1769763957546
1769763957546
Fri Jan 30 2026 14:35:57 GMT+0530 (India Standard Time)
```

Strict Mode in JavaScript

Strict Mode was introduced in **ES5 (ECMAScript 2009)** to make JavaScript code **safer, more secure, and less error-prone**.

It is enabled using the directive:

```
'use strict';
```

It can be applied **globally** (entire script) or **locally** (inside a function).

Why Use Strict Mode?

- **Error Prevention:** Catches common coding mistakes (e.g., undeclared variables).
- **Safer Code:** Prevents accidental globals and disallows unsafe features like with.
- **Future Compatibility:** Disallows usage of reserved keywords meant for future JavaScript versions.

Scope of Strict Mode

- **Global Scope:**
'use strict'; at the top applies to the entire script.

- **Local Scope:**
'use strict'; inside a function applies only to that function.

Key Rules & Restrictions in Strict Mode

- Variables **must be declared** before use.
- Objects and variables **cannot be deleted** directly.
- **Duplicate function parameters** are not allowed.
- **Octal numbers and escape sequences** are forbidden.
- **Reserved keywords** (e.g., public, eval, arguments) cannot be identifiers.
- Cannot modify **read-only properties** or **getter-only properties**.
- this inside a function is **undefined** if no calling object exists.
- **with statement** is not allowed.
- eval() **cannot create variables** in the surrounding scope.
- Certain keywords are **reserved for future use**:
 - implements, interface, package, private, protected

Reserved Keywords in JavaScript

Category	Examples
Control Flow	if, else, switch, while, for, break, continue, return
Declarations	var, let, const, function, class
Object / Context	this, new, instanceof, super
Exception Handling	try, catch, finally, throw
Literals	true, false, null
Others	delete, typeof, void, with, debugger

2. Reserved Keywords Added in ES5 & ES6

ES Version	Keywords
ES5 / ES6	await, class, enum, export, extends, import, let, super

3. Removed but Still Reserved Keywords

(Not supported in many browsers; cannot be used as identifiers)

Keywords
abstract, boolean, byte, char
double, final, float, goto
int, long, native, short
synchronized, throws, transient, volatile

4. JavaScript Built-in Objects

Objects
Array, Object, String, Number, Boolean
Math, Date, JSON, Promise, Map, Set
RegExp, Symbol, Error, Proxy, Reflect
WeakMap, WeakSet, Intl

5. JavaScript Built-in Properties

Properties
length, constructor, prototype
__proto__, caller, callee

6. JavaScript Built-in Methods

Category	Methods
String Methods	toString, split, substring, toUpperCase, trim
Array Methods	push, pop, slice, splice, map, filter, reduce
Search / Match	indexOf, includes, find, match, search

7. Other Reserved Keywords (Window / Browser Objects)

Examples
alert, document, window, navigator
location, history, screen, frames
setTimeout, setInterval, open, close
confirm, prompt, parseInt, parseFloat

8. HTML Event Handlers (Not to Be Used as Identifiers)

Event Handlers
onclick, ondblclick, onmouseover, onmouseout
onmousemove, onkeydown, onkeyup, onkeypress
onfocus, onblur, onchange, onsubmit
onload, onunload, onresize, onscroll

Operators in JavaScript

In JavaScript, an operator is a special symbol that performs an operation on one or more operands (values or variables) and produces a result.

An **operator** in JavaScript is a special symbol used to perform operations on operands (values or variables). Operators help in performing **calculations, comparisons, logical decisions, assignments, and special operations**.

Example : $4 + 5 = 9$

- 4 and 5 → Operands
- + → Operator
- 9 → Result

Types of Operators in JavaScript

JavaScript supports the following types of operators:

1. Arithmetic Operators
2. Comparison Operators
3. Logical (Relational) Operators
4. Bitwise Operators

- 5. Assignment Operators
- 6. Miscellaneous Operators

1. Arithmetic Operators

Used to perform mathematical operations.

Let x=10 and y =20

Operator	Description	Example
+ (Addition)	Adds two operands.	x + y will give 30.
- (Subtraction)	Subtracts the second operand from the first.	x - y will give -10.
* (Multiplication)	Multiplies both operands.	x * y will give 200.
/ (Division)	Divides the numerator by the denominator.	y / x will give 2.
% (Modulus)	Outputs the remainder of an integer division.	y % x will give 0
++ (Increment)	Increases an integer value by one.	x++ will give 11.
-- (Decrement)	Decreases an integer value by one.	x-- will give 9.

2. Comparison Operators

Used to compare two values and return **true or false**.

Let x=10 and y =20

Operator	Description	Example
== (Equal)	Checks if the value of two operands is equal or not. If yes, then the condition becomes true.	(x == y) is false
!= (Not Equal)	Checks if the value of two operands is equal or not. If the values are not equal, then the condition becomes true.	(x != y) is true.
=== (Strict equality)	It checks whether the value and data type of the variable is equal or not. If yes, then the condition becomes true.	(x === y) is false.
!== (Strict inequality)	It checks whether the value and data type of the variable is equal or not. If the values are not equal, then the condition becomes true.	(x !== y) is true.
> (Greater than)	Checks if the value of the left operand is greater than the value of the right operand. If yes, then the condition becomes true.	(x > y) is false.
< (Less than)	Checks if the value of the left operand is less	(x < y) is true.

	than the value of the right operand. If yes, then the condition becomes true.	
$\geq$ (Greater than or Equal to)	Checks if the value of the left operand is greater than or equal to the value of the right operand. If yes, then the condition becomes true.	$(x \geq y)$ is false.
$\leq$ (Less than or Equal to)	Checks if the value of the left operand is less than or equal to the value of the right operand. If yes, then the condition becomes true.	$(x \leq y)$ is true.

3. Logical Operators

Used to perform logical operations.

Operator	Description	Example
$\&\&$ (Logical AND)	If both the operands are non-zero, then the condition becomes true.	$(x \&\& y)$ is false
$\parallel$ (Logical OR)	If any of the two operands are non-zero, then the condition becomes true.	$(x \parallel y)$ is true.
$!$ (Logical NOT)	Reverses the logical state of its operand. If a condition is true, then the Logical NOT operator will make it false.	$!x$ is false.

4. Bitwise Operators

Operate on binary representations of numbers.

Let $x = 2$ and $y = 3$

Operator	Description	Example
$\&$ (Bitwise AND)	It performs a Boolean AND operation on each bit of its integer arguments.	$(x \& y)$ is 2.
$ $ (Bitwise OR)	It performs a Boolean OR operation on each bit of its integer arguments.	$(x y)$ is 3.
$\wedge$ (Bitwise XOR)	It performs a Boolean exclusive OR operation on each bit of its integer arguments. Exclusive OR means that either operand one is true or operand two is true, but not both.	$(x \wedge y)$ is 1.
$\sim$ (Bitwise Not)	It is a unary operator and operates by reversing all the bits in the operand.	$(\sim y)$ is -4.

<< (Left Shift)	It moves all the bits in its first operand to the left by the number of places specified in the second operand. New bits are filled with zeros. Shifting a value left by one position is equivalent to multiplying it by 2, shifting two positions is equivalent to multiplying by 4, and so on.	(x << 1) is 4.
>> (Right Shift)	Binary Right Shift Operator. The left operands value is moved right by the number of bits specified by the right operand.	(x >> 1) is 1.
>>> (Right shift with Zero)	This operator is just like the >> operator, except that the bits shifted in on the left are always zero.	(x >>> 1) is 1.

5. Assignment Operators

Used to assign values to variables.

Operator	Description	Example
= (Simple Assignment)	Assigns values from the right side operand to the left side operand	$z = x + y$ will assign the value of $x + y$ into z
$+=$ (Add and Assignment)	It adds the right operand to the left operand and assigns the result to the left operand.	$z += x$ is equivalent to $z = z + x$
$-=$ (Subtract and Assignment)	It subtracts the right operand from the left operand and assigns the result to the left operand.	$z -= x$ is equivalent to $z = z - x$
$*=$ (Multiply and Assignment)	It multiplies the right operand with the left operand and assigns the result to the left operand.	$z *= x$ is equivalent to $z = z * x$
$/=$ (Divide and Assignment)	It divides the left operand with the right operand and assigns the result to the left operand.	$z /= x$ is equivalent to $z = z / x$
$\%=$ (Modules and Assignment)	It takes modulus using two operands and assigns the result to the left operand.	$z \% x$ is equivalent to $z = z \% x$

Bitwise Assignment

```
let a = 5;
a <<= 1;
console.log(a); // 10
```

6. Miscellaneous Operators

Operator	Description
? : (Conditional)	If Condition is true? Then value X : Otherwise value Y
typeof	It returns the data type of the operand.
?? (Nullish Coalescing Operator)	It returns its right-hand side operand when its left-hand side operand is null or undefined, and otherwise returns its left-hand side operand.
delete	It removes a property from an object.
, (Comma)	It evaluates its operands (from left to right) and returns the value of the last operand.
() (Grouping)	It allows to change the operator precedence.
yield	It is used to pause and resume a generator function.
(Spread)	It is used to expand the iterables such as array or string.
** (Exponentiation)	Raises the left operand to the power of the right operand

Examples:

```
*
Arithmetic Operators
*/
console.log("Arithmetic Operators");
let x = 10;
let y = 20;
console.log(x + y);
console.log(x - y);
console.log(x * y);
console.log(y / x);
console.log(y % x);
x++;
console.log(x);
x--;
console.log(x);

/*String Addition */
console.log("a" + 10);
```

```
console.log(" ");

/*Comparison Operators*/
console.log("Comparison Operators");
let a = 10;
let b = 20;
console.log(a == b);
console.log(a != b);
console.log(a === b); //checks both the value and the type of the operands
console.log(a !== b); //checks if the two values being compared are not equal in either value or data type
console.log(a > b);
console.log(a < b);
console.log(a >= b);
console.log(a <= b);

console.log(" ");

/*Logical Operators*/
console.log("Logical Operators");
let x3 = 10;
let y3 = 0;
console.log(x3 && y3);
console.log(x3 || y3);
console.log(!x3);

console.log(" ");

/*Bitwise Operators*/
console.log("Bitwise Operators");
let x4 = 2;
let y4 = 3;
console.log(x4 & y4);
console.log(x4 | y4);
console.log(x4 ^ y4);
console.log(~y4);
console.log(x4 << 1);
console.log(x4 >> 1);
console.log(x4 >>> 1);

console.log(" ");

/*Assignment Operators*/
```

```
console.log("Assignment Operators");
let z = 10;
z += 5;
console.log(z);
z -= 3;
console.log(z);
z *= 2;
console.log(z);
z /= 4;
console.log(z);
z %= 4;
console.log(z);

console.log(" ");

console.log("Conditional Operator (?:)");
let age = 18;
let result1 = (age >= 18) ? "Eligible" : "Not Eligible";
console.log(result1);

console.log(" ");

console.log("typeof Operator");
console.log(typeof 10);
console.log(typeof "Hello");

console.log(" ");

console.log("Nullish Coalescing Operator (??)");//It returns its right-hand side operand when its left-hand side operand
is null or undefined, and otherwise returns its left-hand side operand
let value = null;
console.log(value ?? "Default");

console.log(" ");

console.log("delete Operator");
let obj = { name: "JS", version: "ES6" };
delete obj.version;
console.log(obj);

console.log(" ");
```



```

console.log("Comma Operator");//It evaluates its operands (from left to right) and returns the value of the last operand.
let result = (1, 2, 3);
console.log(result);

console.log(" ");

console.log("Grouping Operator ( )");
console.log((5 + 3) * 2);

console.log(" ");

console.log("Spread Operator (...");
let arr = [1, 2, 3];
let newArr = [...arr, 4, 5];
console.log(newArr);

console.log(" ");

console.log("Exponentiation Operator (**");
console.log(2 ** 3);

console.log(" ");

console.log("yield Operator (Generator)");
function* demo() {
  yield 1;
  yield 2;
}
let gen = demo();
console.log(gen.next().value);

```

Output:

Arithmetic Operators

30

-10

200

2

0

11

10

a10

Comparison Operators

false

true

false

true

false

true

false

true

Logical Operators

0

10

false

Bitwise Operators

2

3

1

-4

4

1

1

Assignment Operators

15

12

24

6

2

Conditional Operator (?:)

Eligible

typeof Operator

number

string

Nullish Coalescing Operator (??)

Default

delete Operator

{ name: 'JS' }

Comma Operator

3

Grouping Operator ()

16

Spread Operator (...)

[1, 2, 3, 4, 5]

Exponentiation Operator (**)

8

yield Operator (Generator)

1

JavaScript Conditional Statements & Loops

1. if Statement

The if statement checks a condition.

If the condition evaluates to true, the code inside the if block executes.

If it is false, the code is skipped.

Example

```
let age = 20;  
if (age >= 18) {  
    console.log("Eligible to vote");  
}
```

Step-by-Step Explanation

1. The variable age is assigned the value 20.
2. The condition age >= 18 is evaluated.
3. Since 20 >= 18 is true, the statement inside the if block executes.
4. "Eligible to vote" is printed in the console.

Output

Eligible to vote

2. if...else Statement

Executes one block when the condition is true and another when it is false.

This ensures a decision is always made.

Example

```
let age = 15;  
if (age >= 18) {  
    console.log("Eligible to vote");  
} else {  
    console.log("Not eligible to vote");  
}
```

Explanation

1. age is set to 15
2. Condition age >= 18 is checked.
3. Since the condition is false, the else block executes.
4. "Not eligible to vote" is printed.

Output

Not eligible to vote

3. if...else if...else Statement

Used when multiple conditions need to be checked.

- Conditions are checked top to bottom.
- The first true condition is executed.
- Remaining conditions are ignored.

Example

```
let marks = 82;
if (marks >= 90) {
  console.log("Grade A");
} else if (marks >= 75) {
  console.log("Grade B");
} else if (marks >= 50) {
  console.log("Grade C");
} else {
  console.log("Fail");
}
```

Explanation

1. marks value is 82.
2. marks >= 90 → false.
3. marks >= 75 → true.
4. "Grade B" is printed.
5. Remaining conditions are skipped.

Output

Grade B

4.switch Statement

What is a switch Statement?

The switch statement is a **conditional control statement** used to execute **one block of code from multiple possible options**, based on the value of an expression.

It is an **alternative to multiple if...else if statements** when you are comparing **one variable against many fixed values**.

Why use switch?

- Improves **readability** when there are many conditions
- Cleaner and more structured than long if...else if ladders
- Faster to understand and maintain

Syntax

```
switch (expression) {  
  case value1:  
    // statements  
    break;  
  case value2:  
    // statements  
    break;  
  case value3:  
    // statements  
    break;  
  default:  
    // statements  
}
```

Explanation of Syntax

- expression → evaluated once
- case → compares using **strict equality (===)**
- break → exits the switch block
- default → executes if no case matches
- Without break, execution continues to the next case (**fall-through**).

Example

```
let day = 3;  
switch (day) {  
  case 1:  
    console.log("Monday");  
    break;  
  case 2:  
    console.log("Tuesday");  
    break;  
  case 3:  
    console.log("Wednesday");  
    break;  
  case 4:  
    console.log("Thursday");  
    break;  
  case 5:  
    console.log("Friday");  
    break;  
  default:
```

```
    console.log("Invalid day");  
}
```

Step-by-Step Explanation

1. day value is 3
2. switch compares day with each case
3. case 3 matches
4. "Wednesday" is printed
5. break stops further execution

Output

Wednesday

JavaScript Loops

1. while Loop

The while loop executes a block of code repeatedly as long as the condition is true.

It is an entry-controlled loop, meaning the condition is checked before execution.

Example

```
let count = 1;  
while (count <= 5) {  
    console.log("Count:", count);  
    count++;  
}
```

Explanation

1. count starts at 1.
2. Condition count <= 5 is checked.
3. If true, the loop body runs.
4. count is printed.
5. count is incremented by 1.
6. Steps repeat until the condition becomes false.

Output

```
Count: 1  
Count: 2  
Count: 3  
Count: 4  
Count: 5
```

2. do...while Loop

The do...while loop executes the code at least once, even if the condition is false.

This is an exit-controlled loop.

Example

```
let count = 5;
do {
  console.log("Count:", count);
  count++;
} while (count < 5);
```

Explanation

1. The do block executes first without checking the condition.
2. "Count: 5" is printed.
3. count becomes 6.
4. Condition `count < 5` is checked → false.
5. Loop stops.

Output

Count: 5

3. for Loop

The for loop is used when the number of iterations is known.

It combines initialization, condition, and iteration in one line.

Example

```
for (let i = 1; i <= 5; i++) {
  console.log("i =", i);
}
```

Explanation

1. i is initialized to 1.
2. Condition `i <= 5` is checked.
3. If true, loop body executes.
4. Value of i is printed.
5. i is incremented.
6. Loop repeats until condition is false.

Output

```
i = 1
i = 2
i = 3
i = 4
i = 5
```

4. for...in Loop

The for...in loop is used to **iterate over the keys (property names)** of an **object**.

- It **does NOT** give values directly
- It gives **keys (indexes or property names)**
- Mainly used for **objects**
- A normal for loop **cannot directly traverse object properties**, so for...in is introduced

Syntax

```
for (variable in object) {  
  // statements  
}
```

Explanation of Syntax

- variable → Stores the **key** / **property name**
- in → JavaScript operator
- object → Object, array, or string to iterate

In every iteration:

- One **key** is assigned to variable
- Loop continues until all keys are visited

Example : Iterating Object Properties

```
let car = {  
  brand: "OD",  
  model: "Q7",  
  color: "Black"  
};
```

```
for (let key in car) {  
  console.log(key + " -> " + car[key]);  
}
```

How it works

1. key gets "brand" → value is "OD"
2. key gets "model" → value is "Q7"
3. key gets "color" → value is "Black"
4. Loop ends after last property

Output

```
brand -> OD  
model -> Q7  
color -> Black
```

Example 2: for...in with String

In JavaScript, **string is an object**, so it has indexes as keys.

```
let str = "Hello";  
for (let key in str) {  
  console.log(key + " -> " + str[key]);  
}
```

Explanation

- key is the **index**

- `str[key]` is the **character**

Output

0 -> H
1 -> e
2 -> l
3 -> l
4 -> o

Example 3: for...in with Array

Arrays are also objects, and indexes act as keys.

```
let arr = ["Hi", "Hello", 900, true];  
for (let index in arr) {  
  console.log(index + " -> " + arr[index]);  
}
```

Output

0 -> Hi
1 -> Hello
2 -> 900
3 -> true

Example 4: Updating Object Values using for...in

Code

```
let user = {  
  name: "Anila",  
  role: "Developer",  
  active: true  
};  
for (let key in user) {  
  user[key] = null;  
}  
console.log(user);
```

Explanation

- Loop visits each key
- Updates value using `object[key]`

Output

{ name: null, role: null, active: null }

Important Notes about for...in

- Best suited for **objects**
 - Returns **keys**, not values
 - Order is **not guaranteed**
 - Not recommended for arrays when values are needed
-

5.JavaScript for...of Loop

The for...of loop is used to **iterate over values of iterable objects**.

Iterable objects include:

- Arrays
- Strings
- Sets
- Maps
- Unlike for...in, it gives **values directly**, not keys.

Syntax

```
for (variable of iterable) {  
  // statements  
}
```

Explanation

- variable → Current **value**
- of → JavaScript operator
- iterable → Array, string, set, map, etc.

Example 1: for...of with Array

```
let languages = ["JavaScript", "Python", "C", "HTML"];  
for (let lang of languages) {  
  console.log(lang);  
}
```

Output

```
JavaScript  
Python  
C  
HTML
```

Why use for...of here?

- Cleaner syntax
- Direct access to values
- No index needed

Example 2: for...of with String

```
let word = "JavaScript";  
for (let ch of word) {  
  console.log(ch);  
}
```

Output

```
J
```

a
v
a
S
c
r
i
p
T

Example 3: for...of with Set

Set stores **unique values only**

```
let nums = new Set([10, 20, 30, 30, 40]);  
for (let num of nums) {  
  console.log(num);  
}
```

Output

10
20
30
40

Example 4: for...of with Map

Map stores **key-value pairs**

```
let map = new Map();  
map.set("one", 1);  
map.set("two", 2);  
map.set("three", 3);  
for (let [key, value] of map) {  
  console.log(key + " -> " + value);  
}
```

Output

one -> 1
two -> 2
three -> 3

for...in vs for...of (Very Important)

Feature	for...in	for...of
Iterates over	Keys / Indexes	Values

Used mainly for	Objects	Iterables
Array usage	Not recommended	Recommended
Output	Index / Property	Element
Readability	Moderate	High

Exam-Friendly Summary

- for...in → **Object keys**
- for...of → **Iterable values**
- Use for...in for **objects**
- Use for...of for **arrays, strings, sets, maps**

JavaScript Loop Control Statements

JavaScript provides special statements to control the flow of loops. These statements allow you to exit a loop early, skip an iteration, or control nested loops.

1. break Statement

- Immediately terminates (stops) the loop.
- Control jumps outside the loop, even if the loop condition is still true.

Syntax

```
break;
```

Example (using for loop)

```
for (let i = 1; i <= 5; i++) {
  if (i === 3) {
    break;
  }
  console.log(i);
}
```

Output

```
1
2
```

Explanation

- Loop starts from 1.
- When i becomes 3, the break statement executes.
- Loop stops immediately.
- 3, 4, 5 are not executed.

2. continue Statement

- Skips the current iteration of the loop.
- Control moves to the next iteration without stopping the loop.

Syntax

`continue;`

Example

```
for (let i = 1; i <= 5; i++) {  
  if (i === 3) {  
    continue;  
  }  
  console.log(i);  
}
```

Output

1
2
4
5

Explanation

- Loop runs from 1 to 5.
- When i is 3, continue skips that iteration.
- Loop continues normally for remaining values.

String Methods & Properties

1. String length (Property)

Explanation

- Returns the **number of characters** in a string.
- Spaces are also counted.
- It is a **property**, not a method.

Syntax

`string.length`

Example

```
let str = "JavaScript";  
console.log(str.length);
```

Output

10

2. charAt()

Explanation

- Returns the character at a **specific index**.
- Index starts from 0.
- Returns empty string if index is out of range.

Syntax

string.charAt(index)

Example

```
let str = "JavaScript";  
console.log(str.charAt(4));
```

Output

S

3. charCodeAt()

Explanation

- Returns the **UTF-16 code** of a character at a given index.
- Useful for character encoding.

Syntax

string.charCodeAt(index)

Example

```
let str = "ABC";  
console.log(str.charCodeAt(0));
```

Output

65

4. codePointAt()

Explanation

- Returns the **Unicode code point**.
- Supports characters beyond UTF-16 (like emojis).

Syntax

string.codePointAt(index)

Example

```
let str = "😊";
```

```
console.log(str.codePointAt(0));
```

Output

128522

5. concat()

Explanation

- Joins two or more strings.
- Does not modify original string.

Syntax

```
string.concat(str1, str2)
```

Example

```
let a = "Hello";  
let b = "World";  
console.log(a.concat(" ", b));
```

Output

Hello World

6. at()

Explanation

- Returns character at given index.
- Supports **negative indexing**.

Syntax

```
string.at(index)
```

Example

```
let str = "JavaScript";  
console.log(str.at(-1));
```

Output

t

7. Bracket Notation []

Explanation

- Access characters like an array.
- Read-only (cannot modify characters).

Syntax

```
string[index]
```

Example

```
let str = "Hello";  
console.log(str[1]);
```

Output

e

8. slice()

Explanation

- Extracts part of a string.
- Supports negative indexes.

Syntax

```
string.slice(start, end)
```

Example

```
let str = "JavaScript";  
console.log(str.slice(4, 10));
```

Output

Script

9. substring()

Explanation

- Similar to slice()
- Does **not support negative indexes**

Syntax

```
string.substring(start, end)
```

Example

```
let str = "JavaScript";  
console.log(str.substring(0, 4));
```

Output

Java

10. substr() ⚠️ (Deprecated)

Explanation

- Extracts string using **start index + length**
- **Not recommended for new code.**

Syntax

```
string.substr(start, length)
```


Example

```
let str = "JavaScript";  
console.log(str.substr(4, 6));
```

Output

Script

11. toUpperCase()**Explanation**

- Converts string to **uppercase**.

Syntax

```
string.toUpperCase()
```

Example

```
console.log("hello".toUpperCase());
```

Output

HELLO

12. toLowerCase()**Explanation**

- Converts string to **lowercase**.

Syntax

```
string.toLowerCase()
```

Example

```
console.log("HELLO".toLowerCase());
```

Output

Hello

13. isWellFormed()**Explanation**

- Checks if string contains **valid Unicode**.
- Returns true or false.

Syntax

```
string.isWellFormed()
```

Example

```
let str = "Hello";  
console.log(str.isWellFormed());
```

Output

true

14. toWellFormed()

Explanation

- Converts invalid Unicode into valid replacement characters.

Syntax

string.toWellFormed()

Example

```
let str = "Hello";  
console.log(str.toWellFormed());
```

Output

Hello

15. trim()

Explanation

- Removes whitespace from **both ends**.

Syntax

string.trim()

Example

```
let str = " Hello ";  
console.log(str.trim());
```

Output

Hello

16. trimStart()

Explanation

- Removes whitespace from **start only**.

Syntax

string.trimStart()

Example

```
console.log(" Hello".trimStart());
```

Output

Hello

17. trimEnd()

Explanation

- Removes whitespace from **end only**.

Syntax

```
string.trimEnd()
```

Example

```
console.log("Hello  ".trimEnd());
```

Output

```
Hello
```

18. padStart()

Explanation

- Pads string at the **beginning**.

Syntax

```
string.padStart(length, char)
```

Example

```
let num = "5";  
console.log(num.padStart(3, "0"));
```

Output

```
005
```

19. padEnd()

Explanation

- Pads string at the **end**.

Syntax

```
string.padEnd(length, char)
```

Example

```
let str = "JS";  
console.log(str.padEnd(5, "*"));
```

Output

```
JS***
```

20. repeat()

Explanation

- Repeats a string multiple times.

Syntax

```
string.repeat(count)
```

Example

```
console.log("Hi ".repeat(3));
```

Output

```
Hi Hi Hi
```

21. replace()

Explanation

- Replaces **first match only**.

Syntax

```
string.replace(old, new)
```

Example

```
let str = "Hello World";  
console.log(str.replace("World", "JS"));
```

Output

```
Hello JS
```

22. replaceAll()

Explanation

- Replaces **all occurrences**.

Syntax

```
string.replaceAll(old, new)
```

Example

```
let str = "JS is easy. JS is powerful."  
console.log(str.replaceAll("JS", "JavaScript"));
```

Output

```
JavaScript is easy. JavaScript is powerful.
```

23. split()

Explanation

- Converts string into an **array**.

Syntax

```
string.split(separator)
```

Example

```
let str = "HTML,CSS,JS";
```

```
console.log(str.split(","));
```

Output

```
[ 'HTML', 'CSS', 'JS' ]
```

Key Interview Notes

- Strings are **immutable**
 - Methods return **new strings**
 - substr() is deprecated
 - slice() supports negative indexing
 - replace() replaces only first occurrence
-

String Search Methods

String search methods are used to **find characters, words, or patterns** inside a string.

1. indexOf()

Explanation

- Returns the **first occurrence index** of a substring.
- Returns -1 if not found.
- Case-sensitive.

Syntax

```
string.indexOf(searchValue, startIndex)
```

Example

```
let str = "JavaScript is easy";  
console.log(str.indexOf("is"));
```

Output

```
11
```

2. lastIndexOf()

Explanation

- Returns the **last occurrence index** of a substring.
- Searches from right to left.
- Returns -1 if not found.

Syntax

```
string.lastIndexOf(searchValue, startIndex)
```

Example

```
let str = "JS is easy and JS is powerful";  
console.log(str.lastIndexOf("JS"));
```

Output

17

3. search()

Explanation

- Searches using a **regular expression**.
- Returns index of the match.
- Returns -1 if not found.
- Does **not accept start index**.

Syntax

string.search(pattern)

Example

```
let str = "Learning JavaScript";  
console.log(str.search("JavaScript"));
```

Output

9

4. match()

Explanation

- Returns an **array of matches**.
- Can use regular expressions.
- Returns null if no match.

Syntax

string.match(pattern)

Example

```
let str = "JS JS JS";  
console.log(str.match(/JS/g));
```

Output

['JS', 'JS', 'JS']

5. matchAll()

Explanation

- Returns an **iterator of all matches**.
- Must use **global (g) flag**.
- Useful for detailed match information.

Syntax

```
string.matchAll(pattern)
```

Example

```
let str = "JS1 JS2 JS3";  
let result = str.matchAll(/JS\d/g);
```

```
for (let match of result) {  
  console.log(match[0]);  
}
```

Output

```
JS1  
JS2  
JS3
```

6. includes()

Explanation

- Checks if substring exists.
- Returns true or false.

Syntax

```
string.includes(searchValue, startIndex)
```

Example

```
let str = "JavaScript is powerful";  
console.log(str.includes("powerful"));
```

Output

```
true
```

7. startsWith()

Explanation

- Checks if string starts with given value.
- Returns boolean.

Syntax

```
string.startsWith(searchValue, startIndex)
```

Example

```
let str = "JavaScript Tutorial";  
console.log(str.startsWith("JavaScript"));
```

Output

```
true
```

8. endsWith()

Explanation

- Checks if string ends with given value.
- Returns boolean.

Syntax

string.endsWith(searchValue, length)

Example

```
let str = "file.js";  
console.log(str.endsWith(".js"));
```

Output

True

Comparison Table (Important for Interviews)

Method	Returns	Supports RegExp	Multiple Matches
indexOf	Index	✗	✗
lastIndexOf	Index	✗	✗
search	Index	✓	✗
match	Array	✓	✓
matchAll	Iterator	✓	✓
includes	Boolean	✗	✗
startsWith	Boolean	✗	✗
endsWith	Boolean	✗	✗

STRING METHOD SUMMARY

Name	Description
<u>at()</u>	Returns an indexed character from a string
<u>charAt()</u>	Returns the character at a specified index (position)
<u>charCodeAt()</u>	Returns the Unicode of the character at a specified index
<u>codePointAt()</u>	Returns the Unicode value at an index (position) in a string
<u>concat()</u>	Returns two or more joined strings
<u>endsWith()</u>	Returns if a string ends with a specified value
<u>fromCharCode()</u>	Returns Unicode values as characters
<u>includes()</u>	Returns if a string contains a specified value
<u>indexOf()</u>	Returns the index (position) of the first occurrence of a value in a string
<u>isWellFormed()</u>	Returns true if a string is well formed
<u>lastIndexOf()</u>	Returns the index (position) of the last occurrence of a value in a string
<u>length</u>	Returns the length of a string
<u>localeCompare()</u>	Compares two strings in the current locale
<u>match()</u>	Searches a string for a value, or a regular expression, and returns the matches
<u>matchAll()</u>	Searches a string for a value, or a regular expression, and returns the matches
<u>padEnd()</u>	Pads a string at the end
<u>padStart()</u>	Pads a string from the start
<u>prototype</u>	Allows you to add properties and methods to an object
<u>repeat()</u>	Returns a new string with a number of copies of a string
<u>replace()</u>	Searches a string for a pattern, and returns a string where the first match is replaced
<u>replaceAll()</u>	Searches a string for a pattern and returns a new string where all matches are replaced
<u>search()</u>	Searches a string for a value, or regular expression, and returns the index (position) of the match
<u>slice()</u>	Extracts a part of a string and returns a new string

<u>split()</u>	Splits a string into an array of substrings
<u>startsWith()</u>	Checks whether a string begins with specified characters
<u>substr()</u>	Deprecated. Use substring() or slice() instead.
<u>substring()</u>	Extracts characters from a string, between two specified indices (positions)
<u>toLocaleLowerCase()</u>	Returns a string converted to lowercase letters, using the host's locale
<u>toLocaleUpperCase()</u>	Returns a string converted to uppercase letters, using the host's locale
<u>toLowerCase()</u>	Returns a string converted to lowercase letters
<u>toString()</u>	Returns a string or a string object as a string
<u>toUpperCase()</u>	Returns a string converted to uppercase letters
<u>toWellFormed()</u>	Returns a string where "lone surrogates" are replaced with the Unicode replacement character
<u>trim()</u>	Returns a string with removed whitespaces
<u>trimEnd()</u>	Returns a string with removed whitespaces from the end
<u>trimStart()</u>	Returns a string with removed whitespaces from the start
<u>valueOf()</u>	Returns the primitive value of a string or a string object

Example:

```
let str = " JavaScript ES2024 ";
```

```
// at()
console.log(str.at(2));
console.log(str.at(-1));
```

```
// charAt()
console.log(str.charAt(3));
```

```
// charCodeAt()
console.log(str.charCodeAt(0));
```

```
// codePointAt()
console.log(str.codePointAt(0));
```

```
// concat()
console.log("Hello".concat(" ", "World"));

// endsWith()
console.log(str.endsWith("2024 "));

// fromCharCode()
console.log(String.fromCharCode(72, 101, 108, 108, 111));

// includes()
console.log(str.includes("Java"));

// indexOf()
console.log(str.indexOf("a"));

// isWellFormed()
console.log("ABC".isWellFormed());

// lastIndexOf()
console.log(str.lastIndexOf("a"));

// length
console.log(str.length);

// localeCompare()
console.log("apple".localeCompare("banana"));

// match()
console.log("abc123".match(/\d+/));

// matchAll()
let matches = "a1b2c3".matchAll(/\d/g);
for (let m of matches) {
    console.log(m[0]);
}

// padEnd()
console.log("JS".padEnd(6, "*"));

// padStart()
console.log("JS".padStart(6, "*"));

// prototype
String.prototype.reverse = function () {
```

```
    return this.split("").reverse().join("");
};
console.log("hello".reverse());

// repeat()
console.log("Hi ".repeat(3));

// replace()
console.log("I like Java".replace("Java", "JS"));

// replaceAll()
console.log("a-b-a-b".replaceAll("a", "x"));

// search()
console.log("Hello World".search("World"));

// slice()
console.log("JavaScript".slice(4, 10));

// split()
console.log("red,green,blue".split(", "));

// startsWith()
console.log("JavaScript".startsWith("Java"));

// substr() (deprecated)
console.log("JavaScript".substr(4, 6));

// substring()
console.log("JavaScript".substring(4, 10));

// toLocaleLowerCase()
console.log("İSTANBUL".toLocaleLowerCase("tr"));

// toLocaleUpperCase()
console.log("istanbul".toLocaleUpperCase("tr"));

// toLowerCase()
console.log("HELLO".toLowerCase());

// toString()
let s = new String("JS");
console.log(s.toString());

// toUpperCase()
```

```
console.log("hello".toUpperCase());

// toWellFormed()
console.log("\uD800".toWellFormed());

// trim()
console.log(str.trim());

// trimEnd()
console.log(str.trimEnd());

// trimStart()
console.log(str.trimStart());

// valueOf()
let obj = new String("JavaScript");
console.log(obj.valueOf());
```

Output:

```
PS C:\Users\anila\Desktop\Course Study\14. JS> node .\17.string_method.js
```

10

S

74

97

Hello World

t

a

Script

Java

JAVASCRIPT

javascript

true

Hello

JS

JS

JS

005

JS***

Hi Hi Hi

Hello JS

JavaScript is easy. JavaScript is powerful.

['HTML', 'CSS', 'JS']

String Search Methods

0

23

14

['JavaScript', 'JavaScript']

JavaScript

JavaScript

true

true

true

NUMBER METHOD

Name	Description
<u>constructor</u>	Returns the function that created JavaScript's Number prototype
<u>EPSILON</u>	Returns the difference between 1 and the smallest number greater than 1
<u>isFinite()</u>	Checks whether a value is a finite number
<u>isInteger()</u>	Checks whether a value is an integer
<u>isNaN()</u>	Checks whether a value is Number.NaN
<u>isSafeInteger()</u>	Checks whether a value is a safe integer
<u>MAX_SAFE_INTEGER</u>	Returns the maximum safe integer in JavaScript.
<u>MIN_SAFE_INTEGER</u>	Returns the minimum safe integer in JavaScript
<u>MAX_VALUE</u>	Returns the largest number possible in JavaScript
<u>MIN_VALUE</u>	Returns the smallest number possible in JavaScript
<u>NaN</u>	Represents a "Not-a-Number" value
<u>NEGATIVE_INFINITY</u>	Represents negative infinity (returned on overflow)
<u>POSITIVE_INFINITY</u>	Represents infinity (returned on overflow)
<u>parseFloat()</u>	Parses a string and returns a Decimal number

<u>parseInt()</u>	Parses a string and returns a Int number
<u>prototype</u>	Allows you to add properties and methods to an object
<u>toExponential(x)</u>	Converts a number into an exponential notation
<u>toFixed(x)</u>	Formats a number with x numbers of digits after the decimal point
<u>toLocaleString()</u>	Converts a number into a string, based on the locale settings
<u>toPrecision(x)</u>	Formats a number to x length
<u>toString()</u>	Converts a number to a string
<u>valueOf()</u>	Returns the primitive value of a number

JavaScript Functions

1. What is a Function?

A function in JavaScript is a block of reusable code designed to perform a specific task.

Functions help in:

- Code reusability
- Modular programming
- Better readability and maintenance

JavaScript provides many built-in functions like alert() and document.write().

Apart from these, programmers can also create user-defined functions.

2. Function Definition & Declaration

A function must be defined before it is called.

Syntax (Function Declaration)

```
function functionName(parameterList) {
    statements;
}
```

Example

```
function sayHello() {
    alert("Hello there!");
}
```

This is called a **function declaration** or **function statement**.

3. Function Expression

A function expression defines a function as part of an expression and assigns it to a variable. The function name can be omitted, creating an anonymous function.

Syntax

```
const varName = function(parameterList) {  
    statements;  
};
```

Example

```
const myFunc = function (x, y) {  
    return x + y;  
};
```

4. Calling a Function

A function is invoked by writing its name followed by parentheses ().

Example

```
function sayHello() {  
    alert("Hello there!");  
}  
//calling function  
sayHello();
```

5. Function Parameters

Functions can accept parameters to receive values when called.

Example

```
function sayHello(name, age) {  
    console.log(name + " is " + age + " years old.");  
}  
sayHello('Zara', 7);
```

6. Return Statement

The return statement sends a value back to the function caller and terminates function execution.

Example

```
function concatenate(first, last) {  
    return first + last;  
}  
let result = concatenate("Zara ", "Ali");  
alert(result);
```

7. Functions as Variable Values (First-Class Functions)

In JavaScript, functions can be:

- Assigned to variables
- Passed as arguments
- Returned from other functions

Example


```
const myFunc = function () {  
    return "Hello ";  
};  
  
document.getElementById("output").innerHTML = myFunc() + "Users";
```

8. Named Function Expression

A function expression can have a name, but it cannot be called using that name.

Example

```
const sum = function addNumbers(x, y) {  
    return x + y;  
};  
sum(2, 3);    // Valid  
addNumbers(); // Error
```

9. Immediately Invoked Function Expression (IIFE)

An IIFE executes immediately after definition.

Example

```
(function () {  
    alert("Hello World");  
})();
```

10. Function Parameters vs Arguments

- Parameters → Variables in function definition
- Arguments → Actual values passed to the function

Example

```
function mult(a, b, c) {  
    return a * b * c;  
}
```

```
mult(2, 3, 4);
```

11. Arguments Object

The arguments object stores all arguments passed to a function.

Example

```
function merge() {  
    let result = "";  
    for (let i = 0; i < arguments.length; i++) {  
        result += arguments[i] + " ";  
    }  
    return result;  
}
```

```
merge("Hi", "I", "am", "John");
```

12. Passing Arguments by Value

Primitive values are passed by value.

Changes inside the function do not affect original variables.

Example

```
function update(a, b) {  
    a = 20;  
    b = 30;  
}  
let x = 10, y = 15;  
update(x, y);
```

```
console.log(x, y); // 10 15
```

13. Passing Arguments by Reference

Objects are passed by reference.

Changes affect the original object.

Example

```
function update(obj) {  
    obj.domain = "www.tutorialspoint.com";  
}  
let site = { domain: "www.google.com" };  
update(site);  
console.log(site.domain);
```

14. Default Parameters (ES6)

Default parameters provide values when arguments are missing or undefined.

Syntax

```
function functionName(param = defaultValue) {  
}
```

Example

```
function sum(p = 30, q = 40) {  
    return p + q;  
}  
sum(10, 20); // 30  
sum(10);     // 50  
sum();       // 70
```

15. Default Parameters with Expressions

Default values can depend on previous parameters.

Example

```
function sum(p = 2, q = p * 2, r = p * q * 2) {  
    return p + q + r;  
}  
sum(5, 10); // 115
```

JavaScript Object

1. What is a JavaScript Object?

A JavaScript object is a non-primitive data type that allows you to store multiple related values in a single variable using key : value pairs.

- Keys are also called properties
- Values can be:
 - Numbers
 - Strings
 - Booleans
 - Arrays
 - Functions (methods)
 - Other objects

Objects are used to represent real-world entities such as students, employees, cars, users, etc.

2. Why Objects are Needed?

Without objects, we would need many variables:

```
let name = "Anila";  
let age = 22;  
let course = "JavaScript";
```

Using an object:

```
let student = {  
  name: "Anila",  
  age: 22,  
  course: "JavaScript"  
};
```

Advantages:

- Better organization
 - Easier data management
 - Models real-world structures
 - Reduces number of variables
-

3. Structure of an Object

```
const objectName = {  
  key1: value1,  
  key2: value2,
```

```
key3: value3
};
```

4. Creating Objects in JavaScript

4.1 Object Literal (Most Common)

```
const employee = {
  id: 101,
  name: "Ravi",
  salary: 50000
};
```

- ✓ Simple
- ✓ Readable
- ✓ Recommended

4.2 Creating Empty Object and Adding Properties

```
const product = {};  
product.name = "Laptop";  
product.price = 60000;  
Used when values are added later dynamically.
```

4.3 Using new Object()

```
const obj = new Object();  
obj.a = 10;  
Works same as {}, but not recommended in modern JavaScript.
```

5. Accessing Object Properties

5.1 Dot Notation

```
employee.name;  
employee.salary;
```

- ✓ Simple
- ✓ Most commonly used

5.2 Bracket Notation

```
employee["name"];
```

Used when:

- Property name has spaces
 - Property name is stored in a variable
- ```
let key = "salary";
Employee[key];
```
- 

## 6. Modifying Object Properties

### 6.1 Update Property

```
employee.salary = 55000;
```

## 6.2 Add New Property

```
employee.department = "IT";
```

## 6.3 Delete Property

```
delete employee.id;
```

---

## 7. Checking Property Existence

```
"name" in employee;
```

Returns true or false.

---

## 8. Object Methods

When a function is stored inside an object, it is called a method.

```
const user = {
 username: "Anila",
 login: function () {
 return "User logged in";
 }
};
```

Calling method:

```
user.login();
```

---

## 9. The this Keyword in Objects

- this refers to the current object
- Used to access object's own properties

```
const student = {
 name: "Anila",
 greet: function () {
 return "Hello " + this.name;
 }
};
```

### 9.1 this in Different Contexts

```
console.log(this);
```

- In browser → window object

```
function test() {
 console.log(this);
}
```

- Normal function → window
  - Strict mode → undefined
- 

## 10. Nested Objects

Objects can contain other objects.

```
const company = {
 name: "TechSoft",
 address: {
 city: "Mumbai",
 pincode: 400001
 }
};
```

Access:

```
Company.address.city;
```

---

## 11. Looping Through Objects

### 11.1 for...in Loop

```
for (let key in employee) {
 console.log(key, employee[key]);
}
```

Used to iterate over properties.

---

## 12. Built-in Object Methods

### 12.1 Object.keys()

```
Object.keys(employee);
Returns array of keys.
```

### 12.2 Object.values()

```
Object.values(employee);
Returns array of values.
```

### 12.3 Object.entries()

```
Object.entries(employee);
Returns key-value pairs as arrays.
```

---

## 13. Displaying Objects

### 13.1 Using Loop

```
let result = "";
for (let key in employee) {
 result += employee[key] + " ";
}
```

### 13.2 Using JSON.stringify()

```
JSON.stringify(employee);
Converts object into string format.
```

---

## 14. Object Constructor Function

Used to create multiple objects of same type.

```
function Person(name, age) {
 this.name = name;
 this.age = age;
}
```

Creating objects:

```
let p1 = new Person("John", 30);
let p2 = new Person("Anna", 25);
```

---

## 15. Prototype in JavaScript

Prototype allows adding properties and methods to all objects created using a constructor.

### 15.1 Adding Property

```
Person.prototype.country = "India";
```

### 15.2 Adding Method

```
Person.prototype.sayHello = function () {
 return "Hello " + this.name;
};
```

---

## 16. Built-in JavaScript Objects

- Object
  - Array
  - Date
  - Math
  - String
  - Number
  - Boolean
  - RegExp
- 

## 17. Primitive vs Object

| Primitive    | Object          |
|--------------|-----------------|
| Single value | Multiple values |
| Immutable    | Mutable         |
| Fat          | Slightly slower |

## 18. Important Exam Points

- Objects store key-value pairs
- Methods are functions inside objects
- this refers to current object
- Use prototype for shared methods

- Use for...in to iterate objects
- Use JSON.stringify() to display objects

| Topic                     | Explanation                                                                                                                                               | Example                                    |
|---------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------|--------------------------------------------|
| JavaScript Object         | An object is a collection of key–value pairs. Keys are strings, values can be any data type (including functions). Objects represent real-world entities. | <code>{ name: "Anila", age: 25 }</code>    |
| Object Properties         | Properties store data inside an object. They define the characteristics of an object.                                                                     | <code>person.age</code>                    |
| Object Methods            | Methods are functions stored as object properties. They define object behavior.                                                                           | <code>greet: function() {}</code>          |
| Object Literal            | Most common and recommended way to create objects using <code>{}</code> .                                                                                 | <code>const car = { brand: "BMW" };</code> |
| Empty Object              | Object created first, then properties added later.                                                                                                        | <code>const obj = {}; obj.x = 10;</code>   |
| <code>new Object()</code> | Constructor-based object creation (less preferred).                                                                                                       | <code>const obj = new Object();</code>     |
| Dot Notation              | Used to access object properties directly.                                                                                                                | <code>person.name</code>                   |
| Bracket Notation          | Used when property name has spaces or is dynamic.                                                                                                         | <code>person["age"]</code>                 |
| Dynamic Property Access   | Property accessed using a variable name.                                                                                                                  | <code>person[key]</code>                   |
| Add Property              | New property can be added at runtime.                                                                                                                     | <code>person.city = "Delhi";</code>        |
| Update Property           | Existing property value can be changed.                                                                                                                   | <code>person.age = 30;</code>              |
| Delete Property           | Removes a property from an object.                                                                                                                        | <code>delete person.age;</code>            |
| Check Property            | Checks if a property exists in an object.                                                                                                                 | <code>"age" in person</code>               |
| this Keyword              | Refers to the current object calling the method.                                                                                                          | <code>this.name</code>                     |



|                      |                                                                                       |                                     |
|----------------------|---------------------------------------------------------------------------------------|-------------------------------------|
| this in Function     | In non-object functions, this refers to the global object (undefined in strict mode). | function test() {<br>return this; } |
| Nested Objects       | Objects inside another object.                                                        | person.address.city                 |
| for...in Loop        | Used to iterate through object properties.                                            | for(let key in obj)                 |
| Object.values()      | Returns all property values as an array.                                              | Object.values(obj)                  |
| Object.keys()        | Returns all property names as an array.                                               | Object.keys(obj)                    |
| Object.entries()     | Returns key-value pairs as arrays.                                                    | Object.entries(obj)                 |
| JSON.stringify()     | Converts object into JSON string (used for display/storage).                          | JSON.stringify(obj)                 |
| Constructor Function | Blueprint to create multiple similar objects.                                         | function Person() {}                |
| new Keyword          | Used to create object instance from constructor.                                      | new Person("A",20)                  |
| Prototype            | Used to add shared properties/methods to constructor objects.                         | Person.prototype.country="India"    |
| Built-in Objects     | JavaScript provides predefined objects.                                               | Array, Date, Math                   |
| Primitive vs Object  | Primitives store single values; objects store multiple values.                        | number vs object                    |
| Object Reference     | Objects are stored by reference, not value.                                           | obj1 = obj2                         |
| Object Comparison    | Objects are compared by reference, not content.                                       | obj1 === obj2                       |

## ARRAY METHOD

| Method / Property    | Purpose                                                 | Syntax                                 | Modifies Original Array | Returns          | Example Result                             |
|----------------------|---------------------------------------------------------|----------------------------------------|-------------------------|------------------|--------------------------------------------|
| <b>push()</b>        | Adds one or more elements to the <b>end</b> of an array | array.push(item1, item2)               | ✓ Yes                   | New length       | [10,20] → push(30,40) → [10,20,30,40]      |
| <b>pop()</b>         | Removes the <b>last</b> element from an array           | array.pop()                            | ✓ Yes                   | Removed element  | [1,2,3] → pop() → [1,2]                    |
| <b>unshift()</b>     | Adds elements to the <b>beginning</b> of an array       | array.unshift(item1)                   | ✓ Yes                   | New length       | [20,30] → unshift(5,10) → [5,10,20,30]     |
| <b>shift()</b>       | Removes the <b>first</b> element of an array            | array.shift()                          | ✓ Yes                   | Removed element  | [10,20,30] → shift() → [20,30]             |
| <b>slice()</b>       | Extracts part of an array                               | array.slice(start, end)                | ✗ No                    | New array        | [100,200,300,400] → slice(1,3) → [200,300] |
| <b>splice()</b>      | Adds / removes elements                                 | array.splice(start, deleteCount, item) | ✓ Yes                   | Removed elements | [1,2,3,4] → splice(2,1,99) → [1,2,99,4]    |
| <b>indexOf()</b>     | Finds <b>first index</b> of an element                  | array.indexOf(value)                   | ✗ No                    | Index / -1       | ["a","b","c","b"] → indexOf("b") → 1       |
| <b>lastIndexOf()</b> | Finds <b>last index</b> of an element                   | array.lastIndexOf(value)               | ✗ No                    | Index / -1       | lastIndexOf("b") → 3                       |
| <b>includes()</b>    | Checks if element exists                                | array.includes(value)                  | ✗ No                    | true / false     | includes("c") → true                       |
| <b>length</b>        | Counts total elements                                   | array.length                           | ✗ No                    | Number           | [5,10,15,20] → 4                           |

## JavaScript Higher-Order Array Methods

| Method    | Purpose                             | Syntax                 | Modifies Original Array | Returns      | Example Output          |
|-----------|-------------------------------------|------------------------|-------------------------|--------------|-------------------------|
| forEach() | Executes function on each element   | array.forEach(fn)      | ✗ No                    | undefined    | Prints each value       |
| map()     | Transforms elements                 | array.map(fn)          | ✗ No                    | New array    | [10,15,20] → [15,20,25] |
| filter()  | Selects elements based on condition | array.filter(fn)       | ✗ No                    | New array    | Values > 20             |
| reduce()  | Reduces array to single value       | array.reduce(fn, init) | ✗ No                    | Single value | Sum of elements         |

## Spread Operator (...)

| No. | Use Case                  | Description                                                                                    | Syntax Used               | Original Data Modified? | Output / Result                         |
|-----|---------------------------|------------------------------------------------------------------------------------------------|---------------------------|-------------------------|-----------------------------------------|
| 1   | Copying an Array          | Creates a shallow copy of an array. Changes in the new array do not affect the original array. | let a2 = [...a1]          | ✗ No                    | a1 → [1,2,3], a2 → [1,2,3,4]            |
| 2   | Merging Arrays            | Combines multiple arrays into a single new array.                                              | let merged = [...x, ...y] | ✗ No                    | [10,20,30,40]                           |
| 3   | Passing Array to Function | Expands array elements as individual function arguments.                                       | sum(...vals)              | ✗ No                    | 30                                      |
| 4   | Copying an Object         | Creates a shallow copy of an object with the same properties.                                  | let copyUser = {...user}  | ✗ No                    | { name:"Alice", age:25 }                |
| 5   | Merging Objects           | Combines properties of multiple objects into one object.                                       | {...user, ...extra}       | ✗ No                    | { name:"Alice", age:25, city:"London" } |
| 6   | Overriding Properties     | Later properties override earlier ones while copying.                                          | {...user, age:30}         | ✗ No                    | { name:"Alice", age:30 }                |

|   |                         |                                                                         |                     |      |                                             |
|---|-------------------------|-------------------------------------------------------------------------|---------------------|------|---------------------------------------------|
| 7 | Safe Array Modification | Allows modifying array operations without affecting the original array. | [...nums].reverse() | ✗ No | Original [10,20,30],<br>Reversed [30,20,10] |
| 8 | String to Array         | Splits string into array of characters.                                 | [...str]            | ✗ No | ["J","S"]                                   |

## DATE

| Category     | Method / Syntax         | Purpose                     | Return Type | Example / Result      |
|--------------|-------------------------|-----------------------------|-------------|-----------------------|
| Creation     | new Date()              | Creates current date & time | Date object | Current system date   |
|              | new Date("2022-03-25")  | Create date from string     | Date object | Fri Mar 25 2022       |
|              | new Date(y,m,d,h,min,s) | Custom date & time          | Date object | Dec 24 2018 10:33:30  |
|              | new Date(0)             | Epoch time                  | Date object | Jan 01 1970           |
|              | new Date(ms)            | From milliseconds           | Date object | Based on ms           |
| Milliseconds | Positive ms             | After 1970                  | Date        | Future date           |
|              | Negative ms             | Before 1970                 | Date        | Past date             |
| Output       | toString()              | Full date & time            | String      | Human readable        |
|              | toDateString()          | Only date                   | String      | Mon Jan 01 2025       |
|              | toUTCString()           | UTC format                  | String      | GMT based             |
|              | toISOString()           | ISO standard                | String      | YYYY-MM-DDT HH:mm:ssZ |
| Get (Local)  | getFullYear()           | Year                        | Number      | 2025                  |
|              | getMonth()              | Month (0–11)                | Number      | 0 = Jan               |
|              | getDate()               | Day of month                | Number      | 1–31                  |
|              | getDay()                | Day of week                 | Number      | 0 = Sun               |

|             |                      |                      |         |               |
|-------------|----------------------|----------------------|---------|---------------|
|             | getHours()           | Hours                | Number  | 0–23          |
|             | getMinutes()         | Minutes              | Number  | 0–59          |
|             | getSeconds()         | Seconds              | Number  | 0–59          |
|             | getMilliseconds()    | Milliseconds         | Number  | 0–999         |
|             | getTime()            | ms since 1970        | Number  | Timestamp     |
| Get (UTC)   | getUTCFullYear()     | UTC year             | Number  | 2025          |
|             | getUTCMonth()        | UTC month            | Number  | 0–11          |
|             | getUTCDate()         | UTC date             | Number  | 1–31          |
|             | getUTCHours()        | UTC hours            | Number  | 0–23          |
|             | getUTCMinutes()      | UTC minutes          | Number  | 0–59          |
|             | getUTCSeconds()      | UTC seconds          | Number  | 0–59          |
| Static      | Date.now()           | Current timestamp    | Number  | ms value      |
|             | Date.parse(str)      | Date → ms            | Number  | Timestamp     |
| Set Methods | setFullYear(y)       | Set year             | Number  | Modifies date |
|             | setMonth(m)          | Set month            | Number  | 0–11          |
|             | setDate(d)           | Set date             | Number  | 1–31          |
|             | setHours(h,m,s)      | Set time             | Number  | Updates time  |
| Comparison  | date1 > date2        | Compare dates        | Boolean | true / false  |
|             | date1 < date2        | Compare dates        | Boolean | true / false  |
| Locale      | toLocaleString()     | Date + time          | String  | Locale based  |
|             | toLocaleDateString() | Only date            | String  | Locale based  |
|             | toLocaleTimeString() | Only time            | String  | Locale based  |
| Timezone    | getTimezoneOffset()  | UTC difference (min) | Number  | IST = -330    |

# SET

| Category      | Method / Concept                  | Purpose                           | Modifies Original Set | Return Type | Example / Result                 |
|---------------|-----------------------------------|-----------------------------------|-----------------------|-------------|----------------------------------|
| Creation      | <code>new Set(iterable)</code>    | Creates a Set with unique values  | —                     | Set         | <code>[1,2,3,3] → {1,2,3}</code> |
| Uniqueness    | Duplicate values                  | Automatically removed             | —                     | —           | 3 ignored                        |
| Add           | <code>add(value)</code>           | Adds a new value                  | ✓ Yes                 | Set         | <code>add(4)</code>              |
|               | <code>add(duplicate)</code>       | Duplicate ignored                 | ✗ No change           | Set         | <code>add(2)</code> ignored      |
| Size          | <code>size</code>                 | Total elements count              | ✗ No                  | Number      | 3                                |
| Search        | <code>has(value)</code>           | Checks value existence            | ✗ No                  | Boolean     | <code>has(3) → true</code>       |
| Delete        | <code>delete(value)</code>        | Removes specific value            | ✓ Yes                 | Boolean     | <code>{1,2,3} → {1,3}</code>     |
| Clear         | <code>clear()</code>              | Removes all elements              | ✓ Yes                 | undefined   | <code>{}</code>                  |
| Iteration     | <code>forEach(fn)</code>          | Iterates values                   | ✗ No                  | undefined   | Prints values                    |
|               | <code>values()</code>             | Returns value iterator            | ✗ No                  | Iterator    | red, green, blue                 |
|               | <code>keys()</code>               | Same as <code>values()</code>     | ✗ No                  | Iterator    | red, green, blue                 |
|               | <code>entries()</code>            | <code>[value, value]</code> pairs | ✗ No                  | Iterator    | <code>["red","red"]</code>       |
| Data Types    | Mixed values                      | Stores any type                   | —                     | Set         | <code>{1,"one",true}</code>      |
| Conversion    | <code>Spread [...]</code>         | <code>Set → Array</code>          | ✗ No                  | Array       | <code>["red","green"]</code>     |
| Array Cleanup | <code>new Set(array)</code>       | Remove duplicates                 | ✗ No                  | Set         | <code>[1,2,3,4]</code>           |
| Union         | <code>new Set([...a,...b])</code> | Combines two sets                 | ✗ No                  | Set         | <code>{1,2,3,4,5}</code>         |
| Intersection  | <code>filter + has()</code>       | Common elements                   | ✗ No                  | Set         | <code>{3}</code>                 |

|            |                |                        |      |         |       |
|------------|----------------|------------------------|------|---------|-------|
| Difference | !has()         | Elements in A not in B | ✗ No | Set     | {1,2} |
| Type Check | instanceof Set | Confirms Set object    | ✗ No | Boolean | true  |

## JavaScript Classes

| Category             | Keyword / Method  | Purpose                        | Key Rule           | Example / Result        |
|----------------------|-------------------|--------------------------------|--------------------|-------------------------|
| Class Definition     | class ClassName   | Blueprint for creating objects | ES6 feature        | class Student {}        |
| Constructor          | constructor()     | Initializes object properties  | Runs automatically | name, age assigned      |
| Object Creation      | new               | Creates class instance         | Calls constructor  | new Student("Anila",22) |
| Instance Property    | this.property     | Refers to current object       | Instance-specific  | this.name               |
| Instance Method      | method()          | Defines behavior of object     | Access via object  | display()               |
| Method Call          | obj.method()      | Executes class method          | Dot notation       | s1.display()            |
| Inheritance          | extends           | Child class inherits parent    | Enables reuse      | Employee extends Person |
| super()              | super()           | Calls parent constructor       | Must be first line | super(name)             |
| Parent Method Access | Inherited methods | Child uses parent methods      | Automatic          | emp1.greet()            |
| Child Method         | Own method        | Child-specific behavior        | Optional           | showRole()              |
| Static Method        | static            | Belongs to class, not object   | No new required    | MathUtil.add()          |

|                    |                      |                    |                      |                     |
|--------------------|----------------------|--------------------|----------------------|---------------------|
| Static Access      | Class.method()       | Call static method | Object cannot access | MathUtil.multiply() |
| Object Access Rule | Instance → static    | ✗ Not allowed      | Error                | obj.add()           |
| Class Scope        | Methods inside class | Auto strict mode   | Safer code           | ES6 standard        |

## Error Handling

| Topic             | Keyword / Concept | Purpose                | When It Is Used          | Key Rule / Behavior                | Example Result           |
|-------------------|-------------------|------------------------|--------------------------|------------------------------------|--------------------------|
| Error Handling    | try               | Wraps risky code       | When error may occur     | Code executes normally if no error | Executes statements      |
|                   | catch             | Handles runtime errors | When error occurs in try | Receives error object              | Error message printed    |
|                   | error.message     | Error description      | Inside catch block       | Read-only                          | "c is not defined"       |
| try-catch-finally | finally           | Executes always        | Error or no error        | Cannot stop execution              | Cleanup code runs        |
| Runtime Error     | Reference Error   | Variable not declared  | Undefined variable       | Program crashes without catch      | Caught safely            |
| Type Error        | TypeError         | Invalid operation      | Wrong method call        | Must be caught                     | .toUpperCase() on number |
| Debugging         | debugger          | Pause execution        | Development only         | Opens DevTools                     | Step-by-step execution   |
| Custom Error      | throw             | Manually create error  | Validation failures      | Stops execution                    | Throws Error object      |
| Error Object      | new Error()       | Built-in error         | Generic errors           | Has message property               | "Age must be 18"         |



|                     |                           |                         |                     |                             |                   |
|---------------------|---------------------------|-------------------------|---------------------|-----------------------------|-------------------|
| Extending Errors    | class X extends Error     | Custom error type       | App-specific errors | Inherits Error              | ValidationError   |
| Error Name          | this.name                 | Error classification    | Custom errors       | Helps identify error        | "ValidationError" |
| Error Handling Flow | try →<br>throw →<br>catch | Control error execution | Invalid input       | Catch receives thrown error | Graceful handling |